

視窗應用程式開發與常用 元作介紹 (3)

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

常用元件介紹(3) – 容器

常用元件介紹(2)

- 通用元件

1. MaskedTextBox
2. NotifyIcon
3. NumericUpDown
4. ToolTip
5. TreeView

- 容器

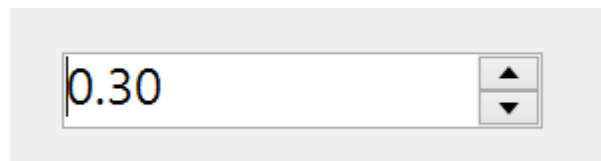
1. Panel
2. FlowLayoutPanel
3. SplitContainer
4. TableControl
5. TableLayoutPanel

- 功能表與工具列

1. ContextMenuStrip
2. ToolStrip
3. ToolStripContainer

NumericUpDown元件常用屬性

- 常用屬性
 - 資料
 - Minimum：允許最小值
 - Maximum：允許最大值
 - Value：當前小值
 - Increment：每次增減值
 - DecimalPlaces：小數位顯示格式
 - 事件
 - ValueChange()：當數值改變時觸發



Timer元件常用屬性與應用

- **Timer**是用來做固定時間間隔觸發的元件
- 常用屬性
 - 行為
 - **Enable**: 是否啟用
 - **Interval**: 時間間隔
 - 事件
 - **Tick()**: 用來放欲觸發的內容

```
private void timer1_Tick(object sender, EventArgs e)
{
    numericUpDown1.Value++;
}
```

TreeView元件常用屬性與應用

- 常用屬性
 - CheckBoxes :核取方塊是否顯示在節點旁
 - ShowLines : 節點間是否顯示連接線
 - Node : 添加節點

treeView1.SelectedNode : 被選擇的節點

```
treeView1.CheckBoxes = true;  
treeView1.ShowLines = true;  
treeView1.Nodes.Add("new 1"); //增加第一層節點  
treeView1.Nodes[0].Nodes.Add("new 2"); //在第一層第一個節點下增加節點  
treeView1.ExpandAll(); //展示所有節點
```



ToolTip

- 用來在滑鼠指向特定元件時顯示指定訊息
- 常用屬性
 - Misc
 - `AutomaticDelay`：顯示訊息時間(單位:ms)
 - `InitialDelay`：停留在元件上多長時間後顯示訊息(單位:ms)
 - `ReshowDelay`：從一元件移到另一元件後顯示訊息的間隔時間(單位:ms)

一般在表單(Form)事件Load中添加

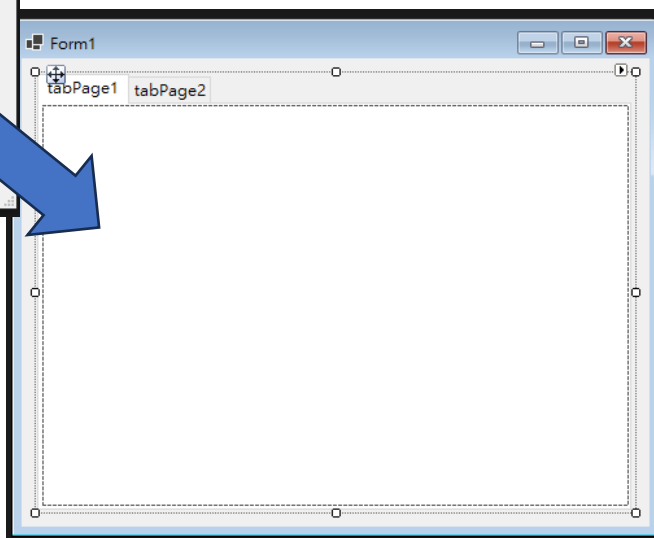
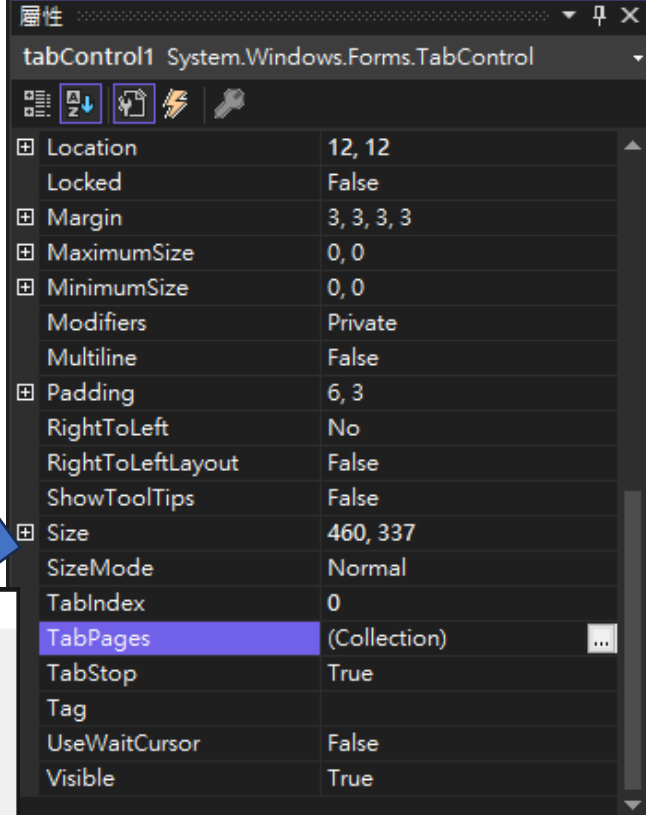
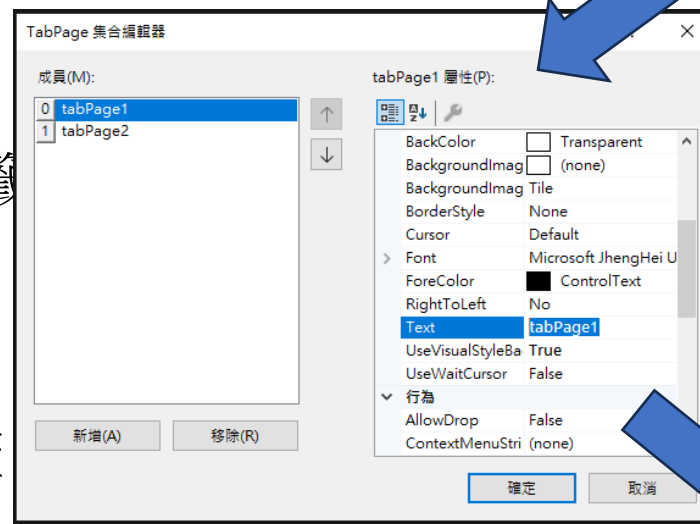
```
private void Form1_Load(object sender, EventArgs e)
{
    toolTip1.SetToolTip(button1, "button1");
    toolTip1.SetToolTip(button1, "button2");
}
```



TabControl元件常用屬性與應用

- 常用成員

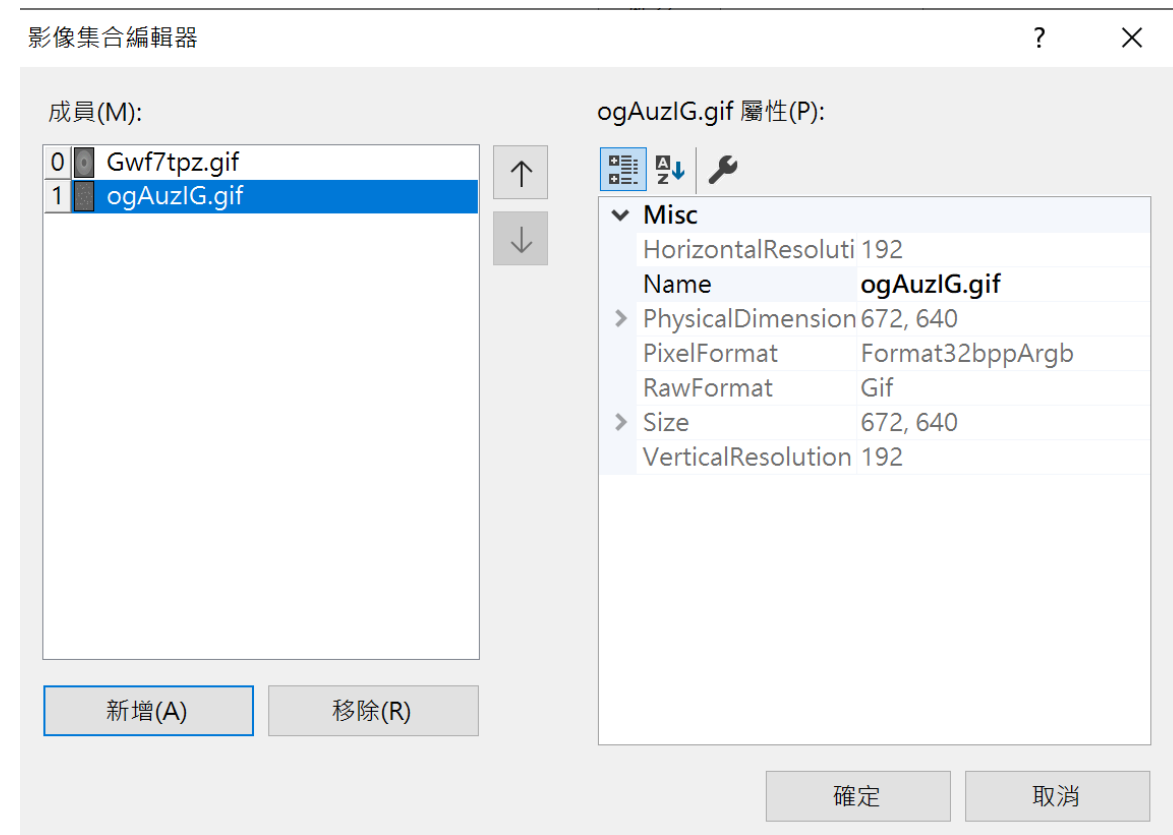
- **TabStop**：是否允許使用者用 Tab 鍵，將焦點放在控制項上。
- **HotTrack**：當滑鼠經過索引標籤頁時，索引標籤是否可變更其外觀。
- **Appearance**：用來設定索引標籤頁的形狀。
- **MultiLine**：是否允許當多個標籤頁一行容納不下時，可否改為多行顯示。
- **Click** 事件：當滑鼠在標籤頁無控制項處點一下，會觸發此事件。
- **SelectedIndexChanged**：當標籤頁切換時會觸發此事件。



ImageList元件常用屬性與應用

- 用來存放多張圖片 供程式使用
- 常用屬性
 - Image: 儲存影像
 - ImageSize: 顯示影像大小
 - TransparentColor: 視為透明的顏色

```
public void addImage()  
{  
    System.Drawing.Image myImage =  
        Image.FromFile  
        (System.Environment.GetFolderPath  
        (System.Environment.SpecialFolder.Personal)  
        + @"\Image.gif");  
    imageList1.Images.Add("myPhoto", myImage);  
    pictureBox1.Image = imageList1.Images[0];  
}
```



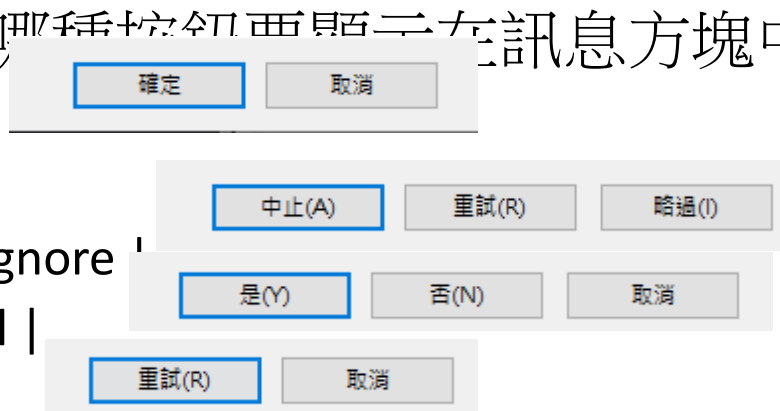
MessageBox

```
MessageBox.Show([Object], Message, Caption, MessageBoxButtons,  
MessageBoxIcon, MessageBoxDefaultButton, NessageBoxOptions);
```

- 參數說明

1. **Object**：在指定物件前面顯示訊息方塊，可省略不寫。
2. **Message**：用來顯示警告或提示使用者的文字。
3. **Caption**：為訊息方塊標題欄上的名稱。
4. **MessageBoxButtons**：用來指定哪種按鈕顯示在訊息方塊中。

- MessageBoxButtons.OK |
- MessageBoxButtons.OKCancel |
- MessageBoxButtons.AbortRetryIgnore |
- MessageBoxButtons.YesNoCancel |
- MessageBoxButtons.YesNo |
- MessageBoxButtons.RetryCancel |



MessageBox

5. **MessageBoxIcon**：指定用哪種小圖示顯示在訊息方塊中

- MessageBoxIcon.Asterisk | 
- MessageBoxIcon.Error | 
- MessageBoxIcon.Exclamation | 
- MessageBoxIcon.Hand | 
- MessageBoxIcon.Information | 
- MessageBoxIcon.None | 無圖示
- MessageBoxIcon.Question | 
- MessageBoxIcon.Stop | 
- MessageBoxIcon.Warning | 

6. **MessageBoxDefaultButton**：指定訊息方塊使用哪個按鈕為預設按鈕。

7. **MessageBoxOptions**：指定訊息方塊使用的顯示及關聯的選項

MessageBox

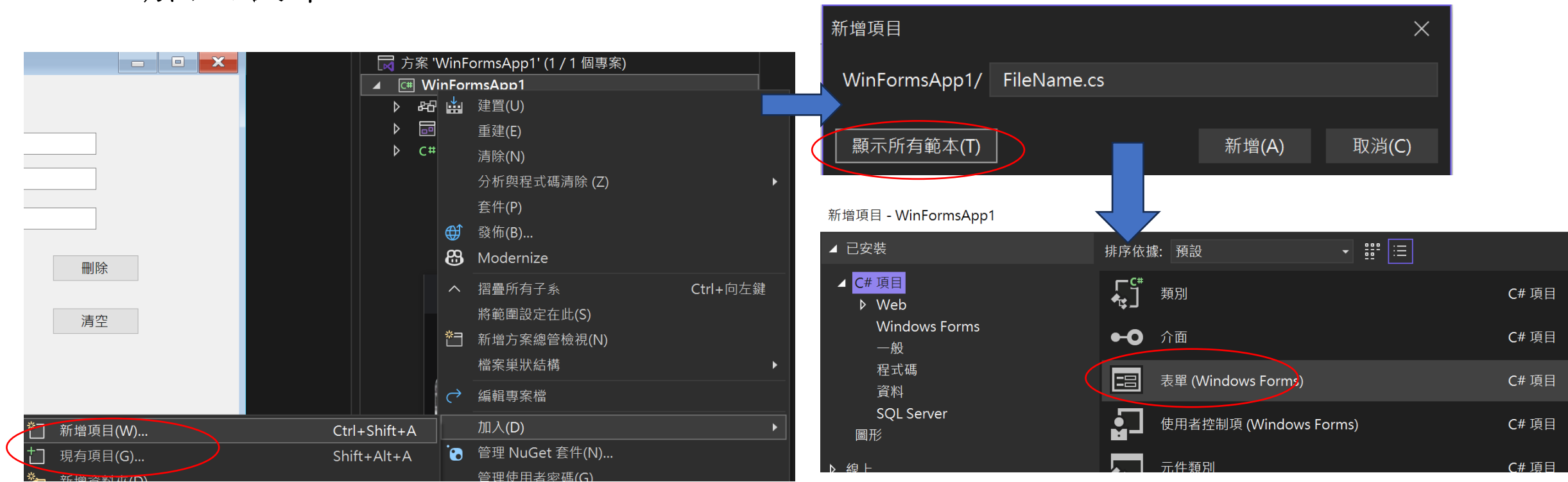
- 傳回值:為按鈕傳回值。當呼叫 `MessageBox.Show` 時，由出現的對話方塊中，在下表中按其中一個按鈕時，會以下表的 `DialogResult` 列舉型別傳回。可宣告一個變數具有 `DialogResult` 型別，再將 `MessageBox.Show` 方法的按鈕結果指定給此變數，並判斷使用者是按下對話方塊的哪個按鈕。

- `DialogResult.OK` | 1 |
- `DialogResult.Cancel` | 2 |
- `DialogResult.Abort` | 3 |
- `DialogResult.Retry` | 4 |
- `DialogResult.Ignore` | 5 |
- `DialogResult.Yes` | 6 |
- `DialogResult.No` | 7 |
- `DialogResult.None` | |

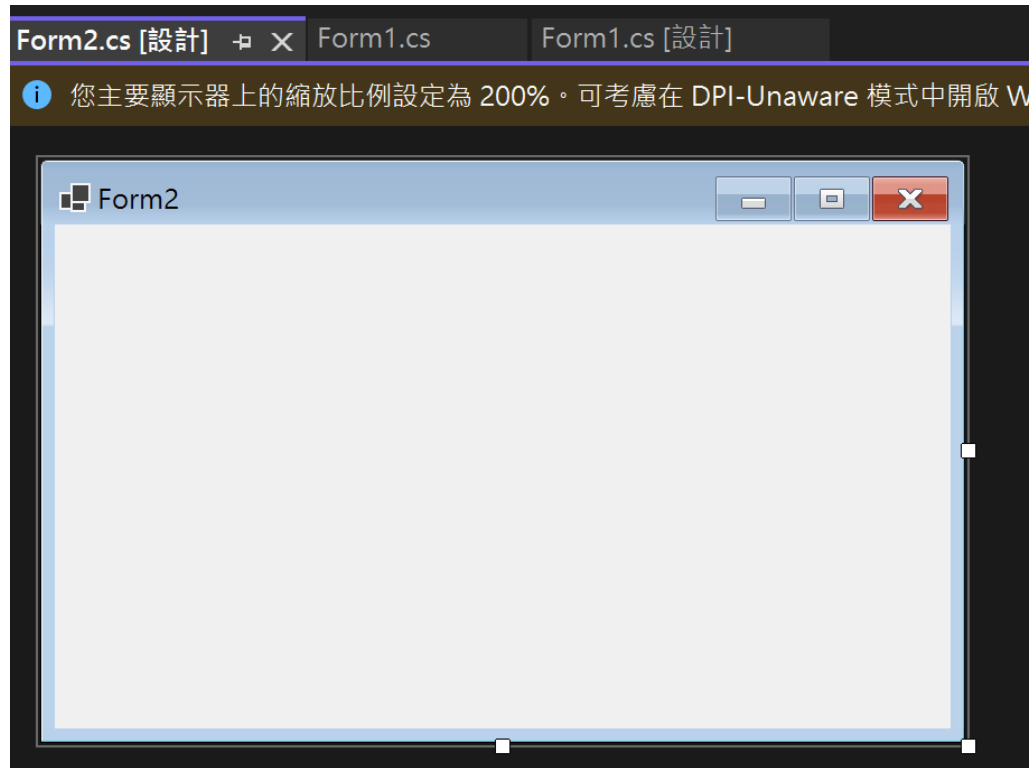
```
DialogResult res;  
res = MessageBox.Show(...);  
if (res == DialogResult.OK)  
{  
    // 按 OK 執行的事  
}  
else  
{  
    // 按其他按鈕執行的事  
}
```

開啟第2個已設計好的視窗(1) – 加入新表單

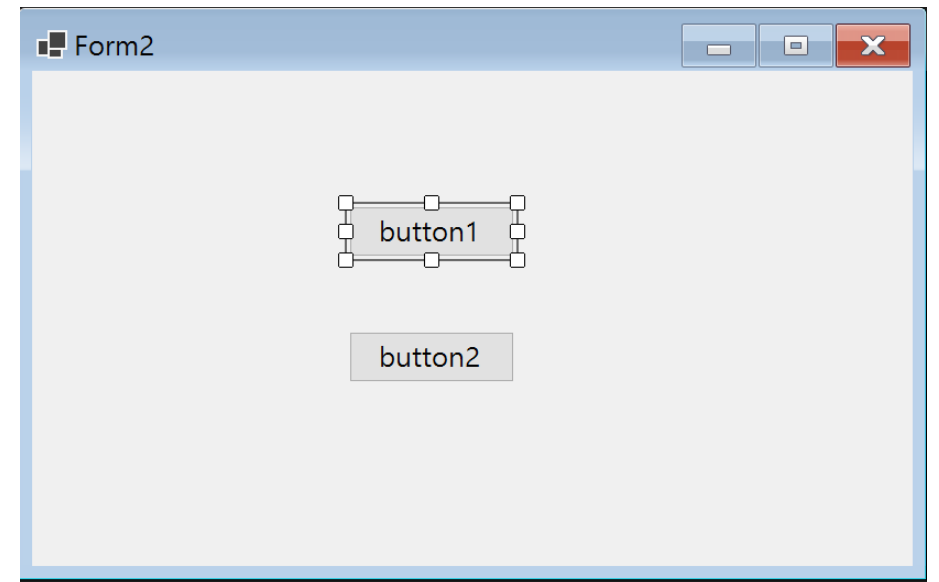
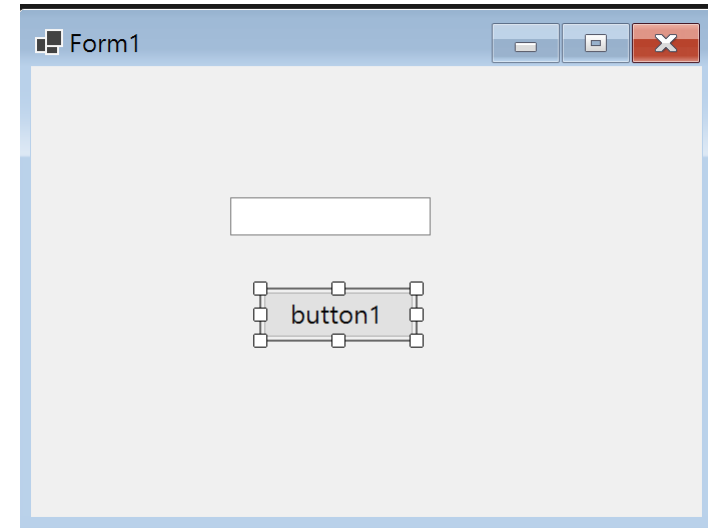
1. 在方案總管中的現有專案以右鍵點選<加入>下的<新增項目>
2. 點選顯示所有範本
3. 加入表單



開啟第2個已設計好的視窗(1) – 設計新表單



設計



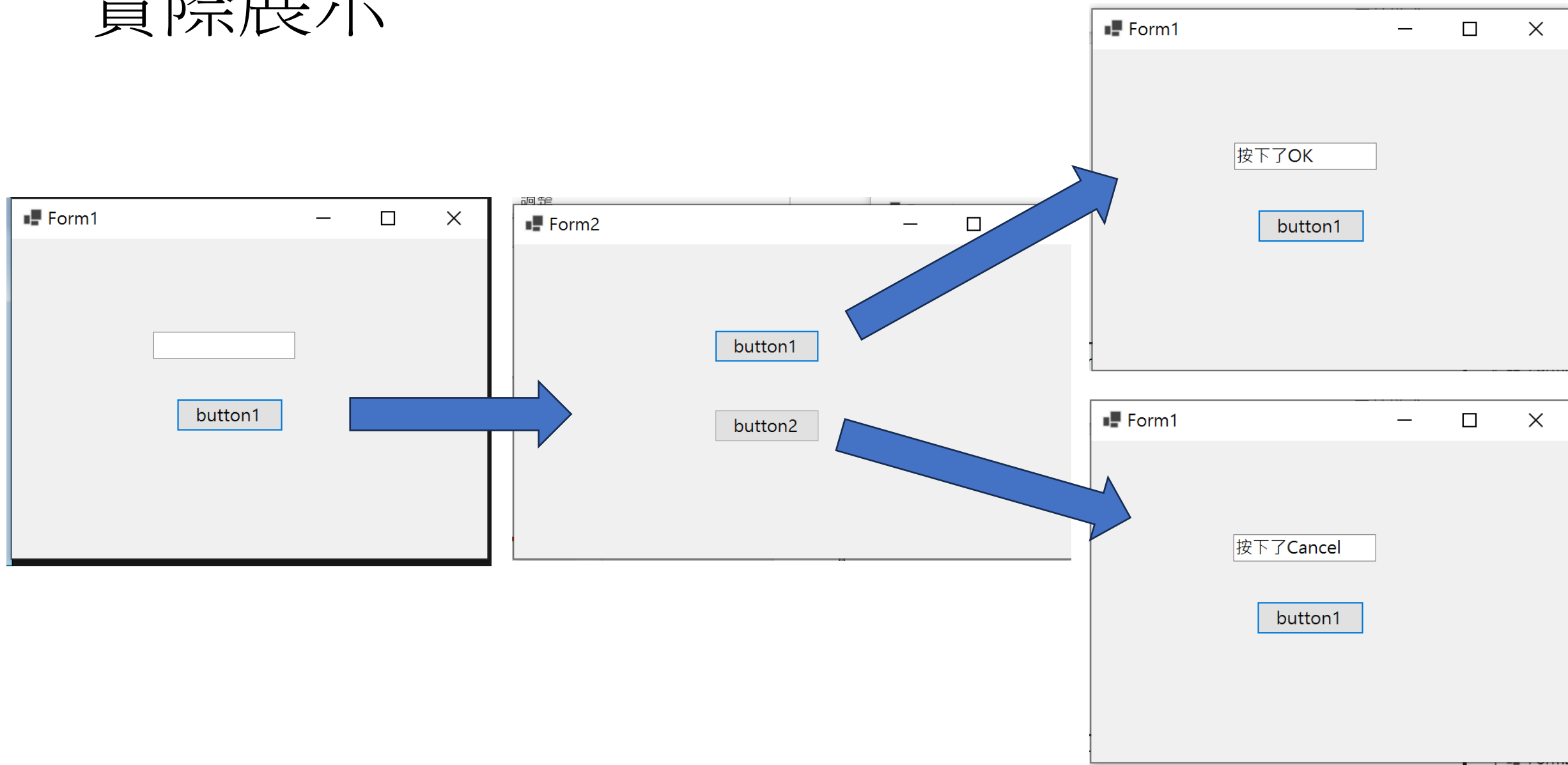
Form 1加入以下程式碼:

```
private void button1_Click(object sender, EventArgs e)
{
    textBox1.Text = ""; //先清空textbox1的文字
    Form2 f = new Form2(); //產生Form2的物件，才可以使用它所提供的Method
    f.ShowDialog(this); //設定Form2為Form1的上層，並開啟Form2視窗。由於在Form1的程式碼內使用this，所以this為Form1的物件本身
    if (f.DialogResult == System.Windows.Forms.DialogResult.OK)
    {
        //若使用者在Form2按下了OK，則進入這個判斷式
        textBox1.Text = "按下了" + f.DialogResult.ToString();
    }
    else if (f.DialogResult == System.Windows.Forms.DialogResult.Cancel)
    {
        //若使用者在Form2按下了Cancel或者直接點選X關閉視窗，都會進入這個判斷式
        textBox1.Text = "按下了" + f.DialogResult.ToString();
    }
    else
    {
        textBox1.Text = "按下了" + f.DialogResult.ToString();
    }
}
```

Form 2加入以下程式碼:

```
public Form2()
{
    InitializeComponent();
    button1.DialogResult = System.Windows.Forms.DialogResult.OK; //設定button1為OK
    button2.DialogResult = System.Windows.Forms.DialogResult.Cancel; //設定button為Cancel
}
```

實際展示



實習時間